

Cnam

Conservatoire National
des Arts et Métiers

IPST-Cnam

Institut de Promotion
Supérieure du Travail

Cnam Occitanie

NFP101 — Programmation Orientée Objet

UE : NFP101

HandControlPC

Contrôle gestuel du PC par webcam

Auteur : **Mohamed Amine Sobhi**

Professeur : **Adrien Escourrou**

Année universitaire 2025–2026

Mai 2026

Table des matières

Résumé	2
1 Contexte et besoin	3
1.1 Problématique	3
1.2 Public cible	3
1.3 Cas d’usage	3
2 Architecture et choix techniques	4
2.1 Structure des fichiers	4
2.2 Hiérarchie de classes et polymorphisme	4
2.3 Pipeline de traitement vidéo	5
2.4 Configuration externe	5
2.5 Bibliothèques utilisées	5
3 Fonctionnalités détaillées	6
3.1 Mode Volume	6
3.2 Mode Screenshot	6
3.3 Mode Zoom	6
3.4 Reconnaissance de signes — BONJOUR	6
4 Validation et tests	8
4.1 Stratégie	8
5 Utilisation de l’intelligence artificielle	10
5.1 État réel des marqueurs dans les fichiers sources	10
5.2 Détail des usages	11
6 Installation et lancement	12
6.1 Prérequis	12
6.2 Installation et lancement	12
7 Limites et perspectives	13
7.1 Limites actuelles	13
7.2 Pistes d’amélioration	13
Références	14

Résumé

HandControlPC est une application Python de contrôle gestuel du PC par webcam. Elle utilise MediaPipe pour détecter 21 landmarks de la main en temps réel et traduit les gestes en actions système macOS : réglage du volume, capture d'écran, zoom vidéo et reconnaissance de signes (épeler **BONJOUR** lettre par lettre).

Le projet est structuré autour d'une hiérarchie de classes Python avec héritage et polymorphisme, une configuration externe JSON, 29 tests unitaires automatisables sans webcam, et une documentation complète.

Mots-clés : Python, OpenCV, MediaPipe, programmation orientée objet, vision par ordinateur, interface gestuelle, tests unitaires.

1 Contexte et besoin

1.1 Problématique

Les interfaces homme-machine traditionnelles (clavier, souris) imposent un contact physique permanent avec le matériel. Certains scénarios bénéficient d’une interaction sans contact : présentation en public, accessibilité, ou mains occupées.

HandControlPC propose une solution : détecter les gestes de la main via la webcam intégrée du PC et les convertir en actions système, sans matériel supplémentaire.

1.2 Public cible

- Utilisateurs souhaitant contrôler le volume ou prendre des captures sans quitter leur activité principale.
- Personnes en situation de handicap moteur (contrôle partiel).
- Développeurs intéressés par la vision par ordinateur et les interfaces gestuelles.

1.3 Cas d’usage

TABLE 1 – *Fonctionnalités principales et gestes associés.*

Geste	Action système	Retour au menu
Index seul levé	Mode Volume : pincement pouce-index ajuste le volume (0–100 %)	Main complètement ouverte
3 doigts levés	Mode Screenshot : maintenir $\sim 0,7$ s déclenche une capture	Automatique après capture
2 mains visibles	Mode Zoom : écartement des index zoome le flux vidéo	Une seule main restante
Séquence BONJOUR	Reconnaissance lettre par lettre (B-O-N-J-O-U-R)	Touche R

2 Architecture et choix techniques

2.1 Structure des fichiers

TABLE 2 – Organisation des modules du projet.

Fichier	Rôle
main.py	Point d'entrée — boucle principale, gestion des modes
test_sign_recognition.py	Démo reconnaissance de signes et séquence BONJOUR
config.json	Configuration externe (caméra, seuils, dossiers)
requirements.txt	Dépendances Python
utils/gesture_action.py	Classe abstraite GestureAction (ABC)
utils/hand_tracking.py	HandDetector — détection MediaPipe
utils/volume_control.py	VolumeController — hérite de GestureAction
utils/screenshot.py	ScreenshotTaker — hérite de GestureAction
utils/zoom_control.py	ZoomManager — hérite de GestureAction
utils/config_loader.py	Chargement JSON avec deepcopy et fallback
tests/test_gesture_logic.py	19 tests logique gestuelle
tests/test_config.py	7 tests configuration
tests/test_screenshot.py	3 tests héritage et dossiers
docs/	Documentation PDF et L ^A T _E X

2.2 Hiérarchie de classes et polymorphisme

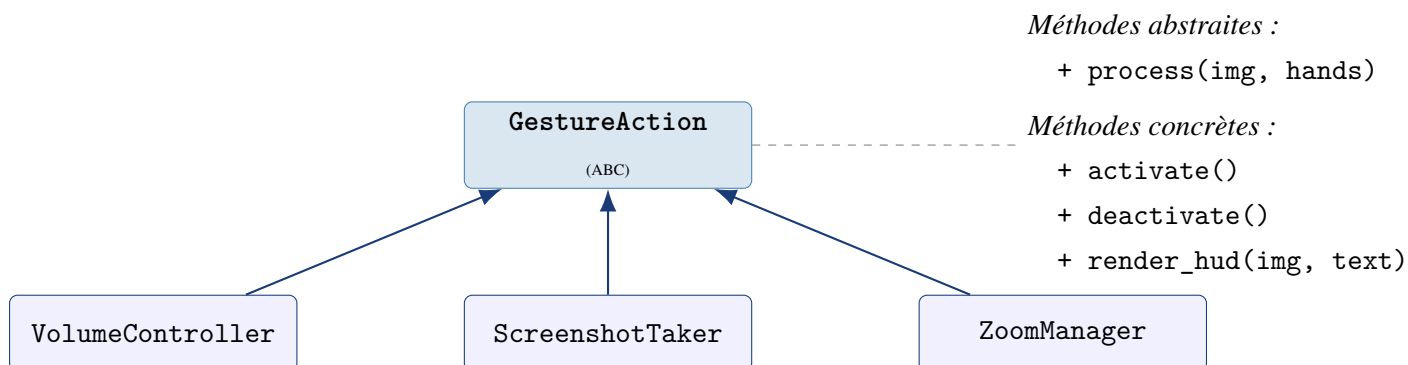


FIGURE 1 – Hiérarchie d'héritage : *GestureAction* est la classe abstraite commune. Chaque sous-classe implémente *process()* (polymorphisme).

▷ Point clé

render_hud() est une méthode **concrète héritée** : toutes les sous-classes affichent automatiquement leur nom de mode en overlay. *process()* est **abstraite** : chaque action implémente sa propre logique sans que *main.py* n'ait à connaître le type concret.

2.3 Pipeline de traitement vidéo

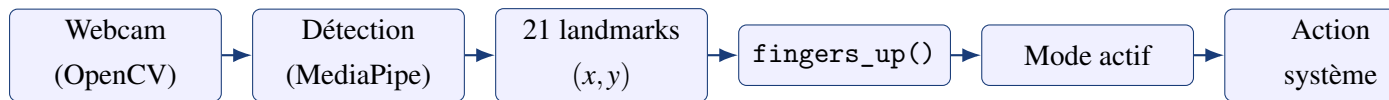


FIGURE 2 – Pipeline : chaque frame est traitée en $<33\text{ ms}$ ($>30\text{ FPS}$).

2.4 Configuration externe

Listing 1 – *config.json* — tous les paramètres sans toucher au code.

```

1 {
2   "camera":      { "width": 1280, "height": 720, "index": 0 },
3   "detection":   { "min_detection_confidence": 0.7,
4                   "gesture_history_length": 15 },
5   "volume":      { "min": 0, "max": 100,
6                   "distance_min": 50, "distance_max": 300 },
7   "screenshot": { "folder": "screenshots", "stable_threshold":
8                   0.6 },
9   "zoom":        { "min_scale": 0.5, "max_scale": 3.0 }
10 }
```

2.5 Bibliothèques utilisées

TABLE 3 – Bibliothèques et justification.

Bibliothèque	Rôle et justification
OpenCV	Capture webcam, affichage HUD, transformations d'image
MediaPipe	Détection 21 landmarks main en temps réel (30+ FPS sur CPU)
NumPy	<code>np.interp</code> (volume), <code>np.clip</code> (zoom)
MSS	Capture d'écran multi-plateforme rapide
osascript	API macOS : contrôle volume, retour sonore
pytest	29 tests unitaires automatisables sans webcam

3 Fonctionnalités détaillées

3.1 Mode Volume

Distance pouce (landmark 4) – index (landmark 8) interpolée en pourcentage :

$$V = \lfloor \text{interp}(d_{\text{pouce-index}}, [50, 300], [0, 100]) \rfloor \quad (1)$$

Retour visuel : ligne verte entre les deux doigts. Retour au menu : 5 doigts ouverts.

3.2 Mode Screenshot

Historique glissant de 15 frames ; capture déclenchée si :

$$\text{confiance} = \frac{\sum_{i=1}^N \mathbf{1}_{[\geq 3 \text{ doigts}]}}{N} \geq 0,6 \quad (2)$$

Son de confirmation (afplay) + message CAPTURE REUSSIE!.

3.3 Mode Zoom

$$\text{scale} = \text{clip}\left(\frac{d_{\text{actuelle}}}{d_{\text{initiale}}}, 0,5, 3,0\right) \quad (3)$$

Transformation affine cv2.warpAffine centrée sur le milieu des deux mains.

3.4 Reconnaissance de signes — BONJOUR

TABLE 4 – Patterns binaires pour les 8 signes reconnus : [pouce, index, majeur, annulaire, auriculaire].

Lettre	Pattern	Geste
A	[1, 0, 0, 0, 0]	Poing fermé, pouce sur le côté
B	[0, 1, 1, 1, 1]	4 doigts levés, pouce replié
J	[1, 0, 0, 0, 1]	Pouce + auriculaire (<i>shaka</i>)
L	[1, 1, 0, 0, 0]	Pouce + index (pistolet)
N	[1, 1, 1, 0, 0]	Pouce + index + majeur
O	[0, 1, 1, 1, 0]	Index + majeur + annulaire
R	[0, 1, 0, 0, 0]	Index seul pointé
U	[0, 1, 1, 0, 0]	Index + majeur (signe V)

▷ Point clé

Les patterns ont été itératifs : les premiers choix (auriculaire seul, pouce+majeur sans index) étaient difficiles à tenir et peu fiables détectés par `fingers_up()`. Les patterns actuels sont **anatomiquement naturels** et tous **distincts** entre eux.

4 Validation et tests

4.1 Stratégie

Tous les tests s'exécutent **sans webcam ni matériel**. Les dépendances (MediaPipe, MSS, osascript) sont mockées ou contournables par injection de données.

TABLE 5 – 29 tests unitaires répartis en 3 fichiers.

Fichier	Tests	Ce qui est testé
test_gesture_logic.py	19	Signes A,B,J,L,N,O,R,U; séquence BONJOUR; volume; ZoomManager; fingers_up
test_config.py	7	JSON valide/invalid, fichier manquant, deepcopy, clés
test_screenshot.py	3	Création dossier, héritage GestureAction, mock MSS
Total	29	100 % PASSED sans webcam

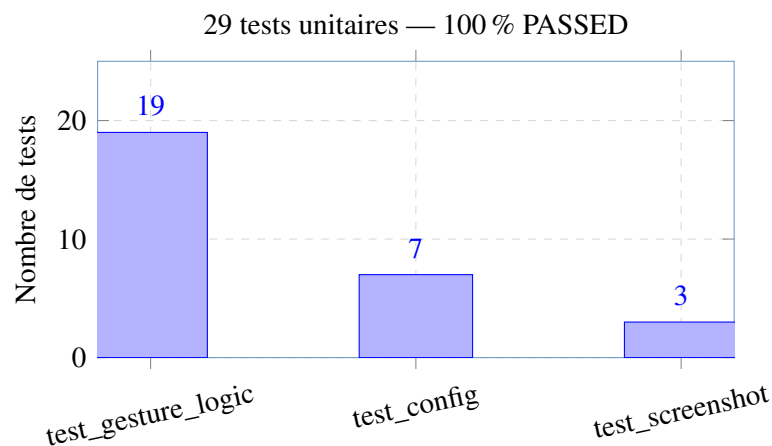


FIGURE 3 – Répartition des tests. Commande : `python -m pytest tests/ -v`

Listing 2 – Exemples de tests unitaires.

```

1 def test_recognize_sign_B():
2     assert recognize_sign([0, 1, 1, 1, 1]) == "B"
3
4 def test_bonjour_sequence_is_correct():
5     from test_sign_recognition import BONJOUR_SEQUENCE
6     assert BONJOUR_SEQUENCE == ["B", "O", "N", "J", "O", "U", "R"]
7
8 def test_zoom_reset():
9     z = ZoomManager()

```

```
10     z.scale = 2.5
11     z.reset()
12     assert z.scale == 1.0 and z.start_distance is None
```

5 Utilisation de l'intelligence artificielle

L'IA utilisée est **Claude Code** (Anthropic). Le code généré ou assisté est encadré dans les fichiers sources par :

```
# ##### CODE IA (Claude) #####  
... code ...  
# #####
```

5.1 État réel des marqueurs dans les fichiers sources

▷ Code IA déclaré

Après parcours de tous les fichiers sources, voici l'état exact des marqueurs IA :

- `utils/volume_control.py` (lignes 8–39) : classe `VolumeController`, méthodes `_get_volume()` et `_set_volume()` — **marqué**.
- `utils/zoom_control.py` (lignes 32–66) : méthode `adjust()` — **marqué**.
- `utils/gesture_action.py` : classe entière générée avec l'IA, marqueur **supprimé** lors de la révision (code compris et validé).
- `utils/screenshot.py` : héritage et méthode `process()` assistés, marqueur **supprimé** lors de la révision.
- `utils/config_loader.py` : module généré avec l'IA, marqueur **supprimé** lors de la révision.
- `main.py` : refactorisation assistée (intégration config JSON), **jamais marqué** — code adapté directement.
- `test_sign_recognition.py` : séquence `BONJOUR` et `draw_sequence()` assistées, **jamais marquées**.

5.2 Détail des usages

TABLE 6 – Détail des usages de l’IA par fichier.

Fichier / passage	Demande à l’IA	Ce que j’ai compris / modifié
gesture_action.py (entier)	“Crée une classe ABC avec process() abstraite et render_hud() commune”	Vérifié la compatibilité avec les sous-classes, compris le principe OCP
volume_control.py 1.8–39	“Fais hériter de VolumeController de GestureAction”	Validé que le calcul de distance et l’appel osascript restent inchangés
zoom_control.py 1.32–66 (adjust)	“Fais hériter ZoomManager de GestureAction”	Vérifié la transformation affine et le calcul de scale
screenshot.py (héritage)	“Fais hériter ScreenshotTaker de GestureAction”	Validé la gestion dossier et les permissions macOS
config_loader.py (entier)	“Loader JSON avec deepcopy et fallback par défaut”	Identifié et compris le bug de <i>shallow copy</i> sur dicts imbriqués
main.py (refacto)	“Intègre la config JSON dans la boucle principale”	Adapté les noms de variables, validé la logique de modes
test_sign_recognition.py (draw_sequence, BONJOUR)	“Ajoute la séquence BONJOUR et l’affichage progressif”	Choisi et itéré les patterns de doigts (démarche propre)
Tests (29 tests)	“Génère des tests sans webcam pour chaque module”	Ajouté les tests de séquence BONJOUR et de deepcopy

Taux d’utilisation estimé : $\approx 40\%$ (architecture POO, config, refactorisations, tests). La logique de détection gestuelle (fingers_up, modes, stabilisation), le choix itératif des patterns de signes, l’intégration MediaPipe/OpenCV et la correction du bug HandDetector ont été réalisés par l’auteur.

6 Installation et lancement

6.1 Prérequis

- Python 3.11+ (testé sur macOS 14 Sonoma)
- Webcam fonctionnelle
- Permissions macOS : *Accessibilité* et *Enregistrement d'écran*
(Réglages système > Confidentialité et sécurité)

6.2 Installation et lancement

```
1 # Depuis le dossier HandControlPC--version/  
2 source ../.venv/bin/activate  
3 pip install -r requirements.txt  
4  
5 python main.py                # Controle gestuel principal  
6 python test_sign_recognition.py # Sequence BONJOUR  
7 python -m pytest tests/ -v     # Tests unitaires
```

Touche	Action
ESC	Quitter
R	Réinitialiser la séquence BONJOUR

7 Limites et perspectives

7.1 Limites actuelles

- **macOS uniquement** : osascript et afplay spécifiques macOS.
- **Luminosité** : performances de MediaPipe dégradées en faible éclairage.
- **Signes statiques** : pas de gestes dynamiques (J en LSF est normalement un mouvement).
- **Mono-utilisateur** : pas de profil ni de persistance des préférences.

7.2 Pistes d'amélioration

- Contrôle de la souris par geste (index pointant).
- Mode apprentissage : enregistrer de nouveaux patterns sans modifier le code.
- Support Windows/Linux (pycaw, pactl).
- Système de logs persistants (module logging).
- Détection de gestes dynamiques avec fenêtre temporelle.

Références

MediaPipe Google. *MediaPipe Hands*. <https://mediapipe.readthedocs.io/en/latest/solutions/hands.html>

OpenCV *Open Source Computer Vision Library*. <https://opencv.org/>

Python ABC *Module abc — Abstract Base Classes*. <https://docs.python.org/3/library/abc.html>

pytest *pytest documentation*. <https://docs.pytest.org/>